PROJECT REPORT TEMPLATE

1. Project Title

Restaurant Billing & Payment

2. Student Details

Enrollment Number: IU2541230457
Project Code: CPP27

3. Problem Statement

Restaurants need an organized way to manage menu items, customer information, orders, bills, and payments. Handling these operations manually can lead to calculation errors, difficulty maintaining records, and delays when retrieving transaction information.

The Restaurant Billing & Payment system addresses this problem by providing a menu-driven C++ application that allows restaurant staff to manage records, create orders, calculate bills, process payments, and generate reports. It also uses file handling to preserve application data between program sessions.

4. Objectives

The main objectives of this project are:

1. To develop a menu-driven restaurant billing application using C++.
2. To manage menu items and customer records through common record operations.
3. To create orders containing menu items and quantities.
4. To calculate bills, including applicable discounts, taxes, and final amounts.
5. To support cash, card, and UPI payment methods, as well as unpaid bills.
6. To apply object-oriented programming concepts such as encapsulation, inheritance, and runtime polymorphism.
7. To demonstrate dynamic memory management and exception handling.
8. To store and retrieve application records using file handling.
9. To generate reports and validate the application's main operations through testing.

5. Features

The application provides the following features:

- **Add Record:** Add supported records to the system.

- **Display Records:** View stored records.
- **Search Record:** Search for existing records.
- **Update Record:** Modify supported record details.
- **Delete Record:** Remove supported records.
- **Main Transaction:** Create an order, select menu items and quantities, calculate a bill, apply a discount, and optionally process payment.
- **Payment Processing:** Handle cash, card, and UPI payments, with an option to pay later.
- **Report:** View summaries of stored records and transactions.
- **Save / Load:** Save application data to files and load previously saved data.
- **Exit:** Close the application.

The program also includes input validation and error handling to help prevent invalid operations.

6. Classes and Responsibilities

- **Responsibility:** Maintains customer records used when creating restaurant orders and bills.

6.3 Order

The Order class represents an order placed by a customer. It contains order lines that record the selected menu items and their quantities.

- **Data members:** Order identification and status information, together with a DynamicArray<OrderLine> that stores the items in the order.
- **Important member functions:** addLine() adds an item to the order, while getSubtotal() calculates the subtotal of the order.
- **Responsibility:** Organizes the items selected by a customer and calculates the order subtotal.

6.4 OrderLine

The OrderLine class represents one item entry within an order.

- **Data members:** Menu item ID, item name, unit price, and quantity.
- **Important member functions:** Functions used to access the stored item details and calculate the line amount.
- **Responsibility:** Stores the details of each ordered item. Keeping the item name and price in the order line preserves the details used for that transaction.

6.5 Bill

The Bill class represents the bill generated for an order.

- **Data members:** Bill ID, order ID, customer ID, subtotal and calculated amounts, discount percentage, date, and payment status.
- **Important member functions:** markPaid() updates the bill's payment status, and markCancelled() marks a bill as cancelled. Other functions support accessing bill information and calculated amounts.
- **Responsibility:** Maintains billing information, including the discount, tax, total, and payment status.

6.6 Payment

The Payment class is an abstract base class that provides a common interface for payment operations.

- **Data members:** Common payment information as defined by the base class.
- **Important member functions:** Virtual functions provide the common operations implemented by the different payment types.
- **Responsibility:** Defines the common structure for recording payments and enables the program to process different payment methods through a shared interface.

The project includes three derived classes:

- **CashPayment:** Handles payments made in cash.
- **CardPayment:** Handles card payments.
- **UpiPayment:** Handles payments made through UPI.

These derived classes demonstrate inheritance and runtime polymorphism. The program can work with the common Payment interface while using the appropriate payment implementation.

6.7 DynamicArray

The DynamicArray<T> class is a template class used to manage collections of objects.

- **Data members:** Internal storage and capacity-related information required to manage the collection dynamically.
- **Important member functions:** Operations for managing and accessing elements in the collection.

- **Responsibility:** Allows collections, such as order lines and application records, to grow as required instead of relying only on a fixed-size array.

6.8 FileManager

The FileManager class handles the reading and writing of application data files.

- **Data members:** File-management information required by the implementation.
- **Important member functions:** readLines() reads lines from a file, and writeLines() writes lines to a file.
- **Responsibility:** Provides the file operations used to preserve menu, customer, order, order-item, bill, payment, and metadata records.

6.9 RestaurantSystem

The RestaurantSystem class coordinates the main operations of the application.

- **Data members:** Collections of menu items, customers, orders, bills, and payment records, along with the file manager and application state.
- **Important member functions:** Functions for placing orders, collecting payments, paying existing bills, saving files, and managing the application's operations.
- **Responsibility:** Connects the user interface with the entity classes and file-handling functionality. It coordinates transactions, record management, reporting, and data persistence.

Overall, the classes divide the application into separate responsibilities. The entity classes store restaurant information, DynamicArray<T> manages collections, Payment and its derived classes demonstrate inheritance and polymorphism, FileManager handles file operations, and RestaurantSystem coordinates the complete application.

7. OOP Concepts Used

The Restaurant Billing & Payment System demonstrates several important concepts of Object-Oriented Programming (OOP) in C++. These concepts help organize the program into reusable classes and make the application easier to maintain.

7.1 Classes and Objects

A class defines the data and functions associated with a particular entity. The project uses classes such as MenuItem, Customer, Order, Bill, and Payment to represent the main components of a restaurant billing system.

Objects created from these classes represent individual menu items, customers, orders, bills, and payment records. The RestaurantSystem class coordinates the operations performed on these objects.

7.2 Encapsulation

Encapsulation combines data and the functions that operate on that data within a class. The project uses classes to organize information and provide member functions for accessing or modifying it.

For example, the MenuItem class provides setter functions such as setName(), setCategory(), setPrice(), and setAvailable() to update menu item details. This approach keeps related operations within the class.

7.3 Constructors and Destructors

Constructors initialize objects when they are created. The project uses constructors to initialize objects with the information required by their respective classes.

Destructors are used to perform cleanup when objects are destroyed. They are particularly important when a class manages dynamically allocated memory or other resources.

7.4 Inheritance

Inheritance allows a derived class to reuse and extend the functionality of a base class. In this project, CashPayment, CardPayment, and UpiPayment are derived from the abstract Payment class.

Each derived class represents a different payment method while sharing the common payment interface defined by the base class.

7.5 Runtime Polymorphism

Runtime polymorphism allows the appropriate derived-class implementation to be selected through a base-class pointer or reference when virtual functions are used.

The project uses a common Payment interface for cash, card, and UPI payments. This allows the system to handle different payment types through the same general interface while invoking the appropriate implementation.

7.6 Dynamic Memory Management

The project uses dynamic memory management to handle collections and payment objects. The DynamicArray<T> template manages collections that can grow as needed, while payment objects can be managed through pointers.

Dynamically allocated memory must be released correctly when it is no longer required. Appropriate cleanup helps prevent memory leaks and supports reliable program execution.

7.7 Exception Handling and Validation

Input validation helps prevent invalid information from being accepted by the application. The system validates relevant inputs, such as prices, quantities, discounts, and payment details.

Exception handling is used in relevant operations to handle invalid values and communicate errors. For example, invalid payment details can be rejected rather than being processed as a valid payment.

7.8 Summary

The use of classes, encapsulation, constructors and destructors, inheritance, runtime polymorphism, dynamic memory management, and exception handling demonstrates the practical application of OOP concepts in a C++ project. These concepts help make the Restaurant Billing & Payment System modular and easier to maintain.

8. File Handling

The Restaurant Billing & Payment System uses file handling in C++ to store application records in plain-text files. This allows the program to preserve information between executions without requiring a database server.

8.1 Data Files Used

The application stores its data in the Data directory. The following files are used:

- **menu.txt:** Stores restaurant menu item records, including item details and availability.
- **customers.txt:** Stores customer records and their details.
- **orders.txt:** Stores order information.

- **order_items.txt:** Stores the individual items and quantities associated with orders.
- **bills.txt:** Stores bill information, including calculated amounts and payment status.
- **payments.txt:** Stores payment records and payment-related details.
- **meta.txt:** Stores metadata used by the application, including information needed to maintain record identifiers.

8.2 Reading and Writing Files

The project uses the FileManager class to perform file-reading and file-writing operations. Its readLines() function reads lines from a file, while writeLines() writes lines to a file.

The RestaurantSystem class coordinates loading and saving the application's records. When the program starts, it loads the existing records from the data files. After operations modify the data, the system can save the updated records to the corresponding files.

8.3 Data Persistence

Data persistence means that information remains available after the application is closed and started again. In this project, persistence is achieved by storing application records in text files.

For example, menu item details, customer records, orders, bills, and payments can be loaded from the Data directory when the program starts. This avoids having to enter all the existing information again during every execution.

8.4 Persistence Testing

The application was tested by building and launching the executable from the project directory. During startup, the program reported that it had loaded six menu items, three customers, four orders, four bills, and three payments from the Data directory.

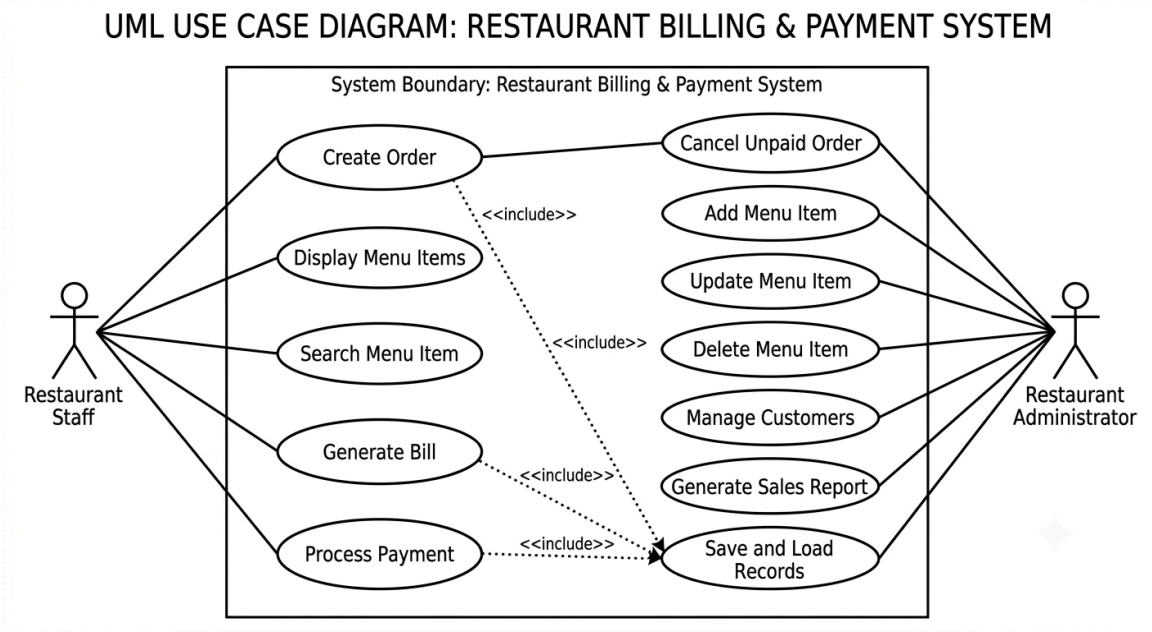
This confirmed that the application could read the existing records during startup. To complete the persistence test before submission, a change should be saved, the program should be closed and reopened, and the saved change should be checked again.

8.5 Importance of File Handling

File handling provides a simple way to maintain records between executions. Separating the data files from the source code also makes it easier to inspect stored information and

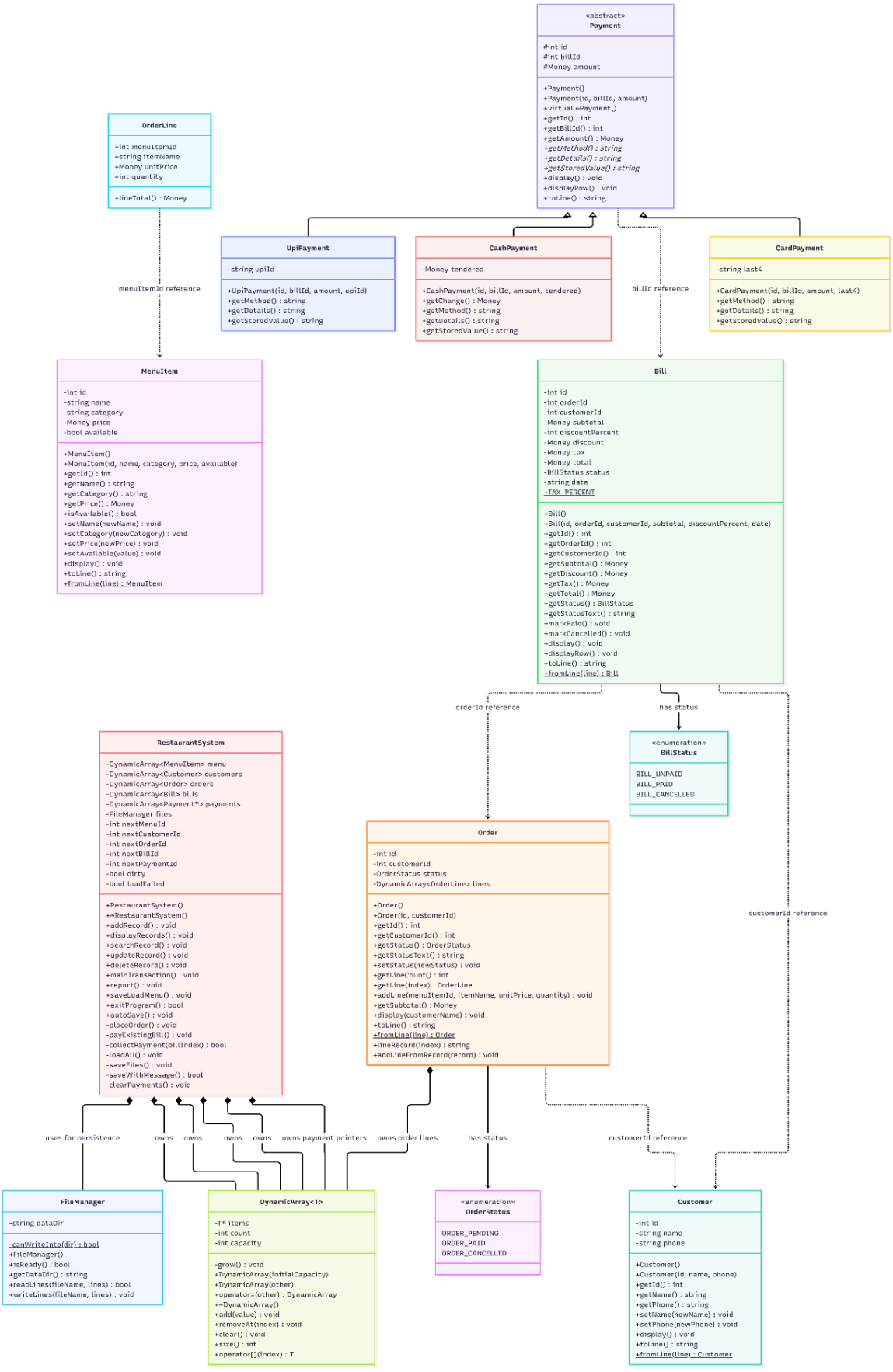
manage the application's records. The project demonstrates how C++ file operations can support a practical record-management application without a database.

9. UML Diagrams



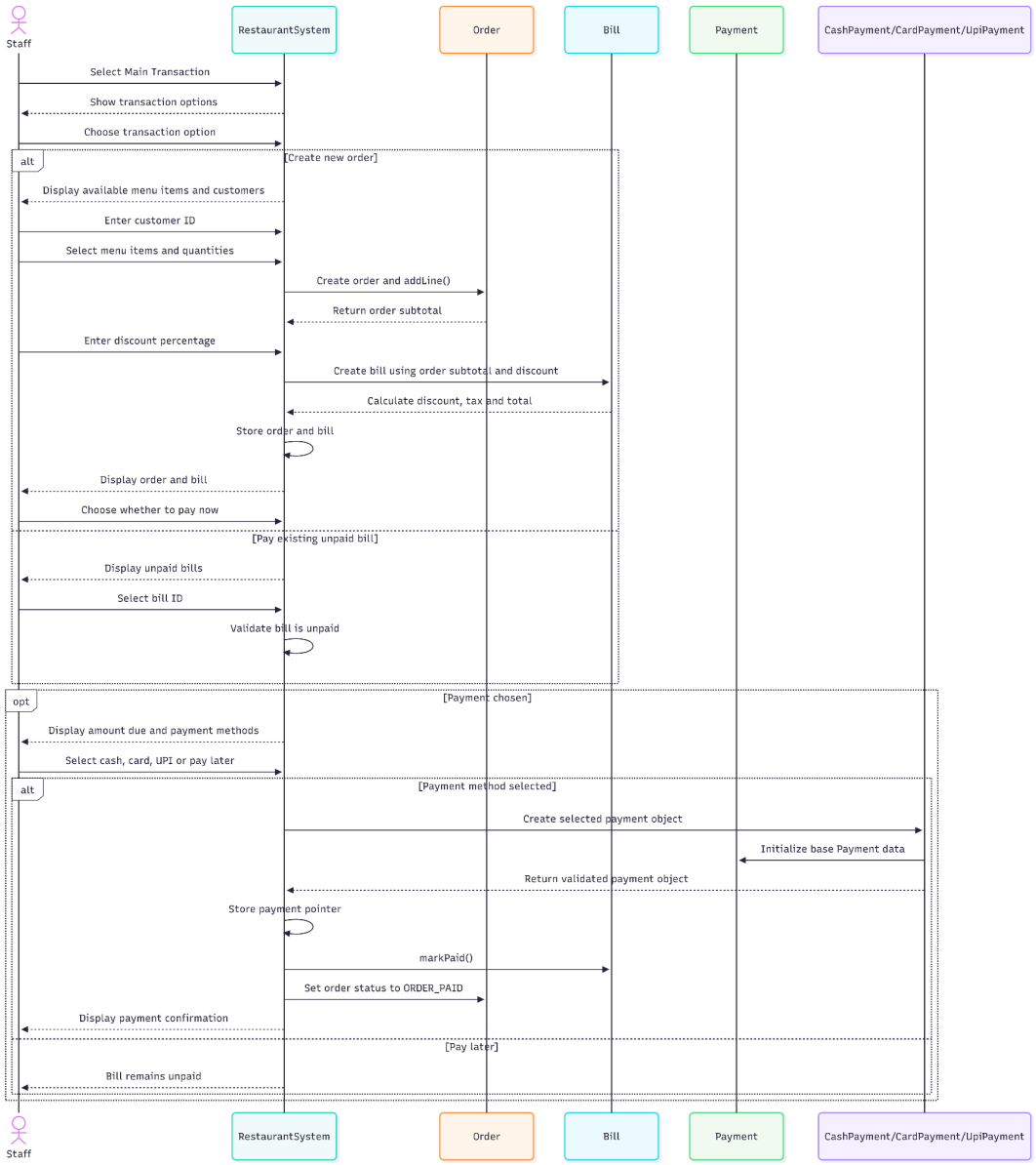
9.1 Use Case Diagram

Shows the actors who interact with the system and the operations they can perform, such as managing records, processing transactions, and generating reports.


«abstract»
Payment
#int id
#int billId
#Money amount
+Payment()
+Payment(id, billId, amount)
+virtual ~Payment()
+getId() : int
+getBillId() : int
+getAmount() : Money
+getMethod() : string
+getDetails() : string
+getStoredValue() : string
+display() : void
+displayRow() : void
+toLine() : string
OrderLine
+int menuItemId
+string itemName
+Money unitPrice
+int quantity
+lineTotal() : Money
UpiPayment
-string upiId
+UpiPayment(id, billId, amount, upiId)
+getMethod() : string
+getDetails() : string
+getStoredValue() : string
CashPayment
-Money tendered
+CashPayment(id, billId, amount, tendered)
+getChange() : Money
+getMethod() : string
+getDetails() : string
+getStoredValue() : string
CardPayment
-string last4
+CardPayment(id, billId, amount, last4)
+getMethod() : string
+getDetails() : string
+getStoredValue() : string
menuItemId reference
billId reference
MenuItem
-int id
-string name
-string category
-Money price
-bool available
+MenuItem()
+MenuItem(id, name, category, price, available)
+getId() : int
+getName() : string
+getCategory() : string
+getPrice() : Money
+isAvailable() : bool
+setName(newName) : void
+setCategory(newCategory) : void
+setPrice(newPrice) : void
+setAvailable(value) : void
+display() : void
+toLine() : string
+fromLine(line) : MenuItem
Bill
-int id
-int orderId
-int customerId
-Money subtotal
-int discountPercent
-Money discount
-Money tax
-Money total
-BillStatus status
-string date
+TAX_PERCENT
+Bill()
+Bill(id, orderId, customerId, subtotal, discountPercent, date)
+getId() : int
+getOrderId() : int
+getCustomerId() : int
+getSubtotal() : Money
+getDiscount() : Money
+getTax() : Money
+getTotal() : Money
+getStatus() : BillStatus
+getStatusText() : string
+markPaid() : void
+markCancelled() : void
+display() : void
+displayRow() : void
+toLine() : string
+fromLine(line) : Bill
orderId reference
has status
«enumeration»
BillStatus
BILL_UNPAID
BILL_PAID
BILL_CANCELLED
RestaurantSystem
-DynamicArray<MenuItem> menu
-DynamicArray<Customer> customers
-DynamicArray<Order> orders
-DynamicArray<Bill> bills
-DynamicArray<Payment*> payments
-FileManager files
-int nextMenuId
-int nextCustomerId
-int nextOrderId
-int nextBillId
-int nextPaymentId
-bool dirty
-bool loadFailed
+RestaurantSystem()
+~RestaurantSystem()
+addRecord() : void
+displayRecords() : void
+searchRecord() : void
+updateRecord() : void
+deleteRecord() : void
+mainTransaction() : void
+report() : void
+saveLoadMenu() : void
+exitProgram() : bool
+autoSave() : void
-placeOrder() : void
-payExistingBill() : void
-collectPayment(billIndex) : bool
-loadAll() : void
-saveFiles() : void
-saveWithMessage() : bool
-clearPayments() : void
Order
-int id
-int customerId
-OrderStatus status
-DynamicArray<OrderLine> lines
+Order()
+Order(id, customerId)
+getId() : int
+getCustomerId() : int
+getStatus() : OrderStatus
+getStatusText() : string
+setStatus(newStatus) : void
+getLineCount() : int
+getLine(index) : OrderLine
+addLine(menuItemId, itemName, unitPrice, quantity) : void
+getSubtotal() : Money
+display(customerName) : void
+toLine() : string
+fromLine(line) : Order
+lineRecord(index) : string
+addLineFromRecord(record) : void
customerId reference
uses for persistence
owns
owns
owns
owns
owns payment pointers
owns order lines
has status
customerId reference
FileManager
-string dataDir
-canWriteInto(dir) : bool
+FileManager()
+isReady() : bool
+getDataDir() : string
+readLines(fileName, lines) : bool
+writeLines(fileName, lines) : void
DynamicArray<T>
-T* items
-int count
-int capacity
-grow() : void
+DynamicArray(initialCapacity)
+DynamicArray(other)
+operator=(other) : DynamicArray
+~DynamicArray()
+add(value) : void
+removeAt(index) : void
+clear() : void
+size() : int
+operator[](index) : T
«enumeration»
OrderStatus
ORDER_PENDING
ORDER_PAID
ORDER_CANCELLED
Customer
-int id
-string name
-string phone
+Customer()
+Customer(id, name, phone)
+getId() : int
+getName() : string
+getPhone() : string
+setName(newName) : void
+setPhone(newPhone) : void
+display() : void
+toLine() : string
+fromLine(line) : Customer

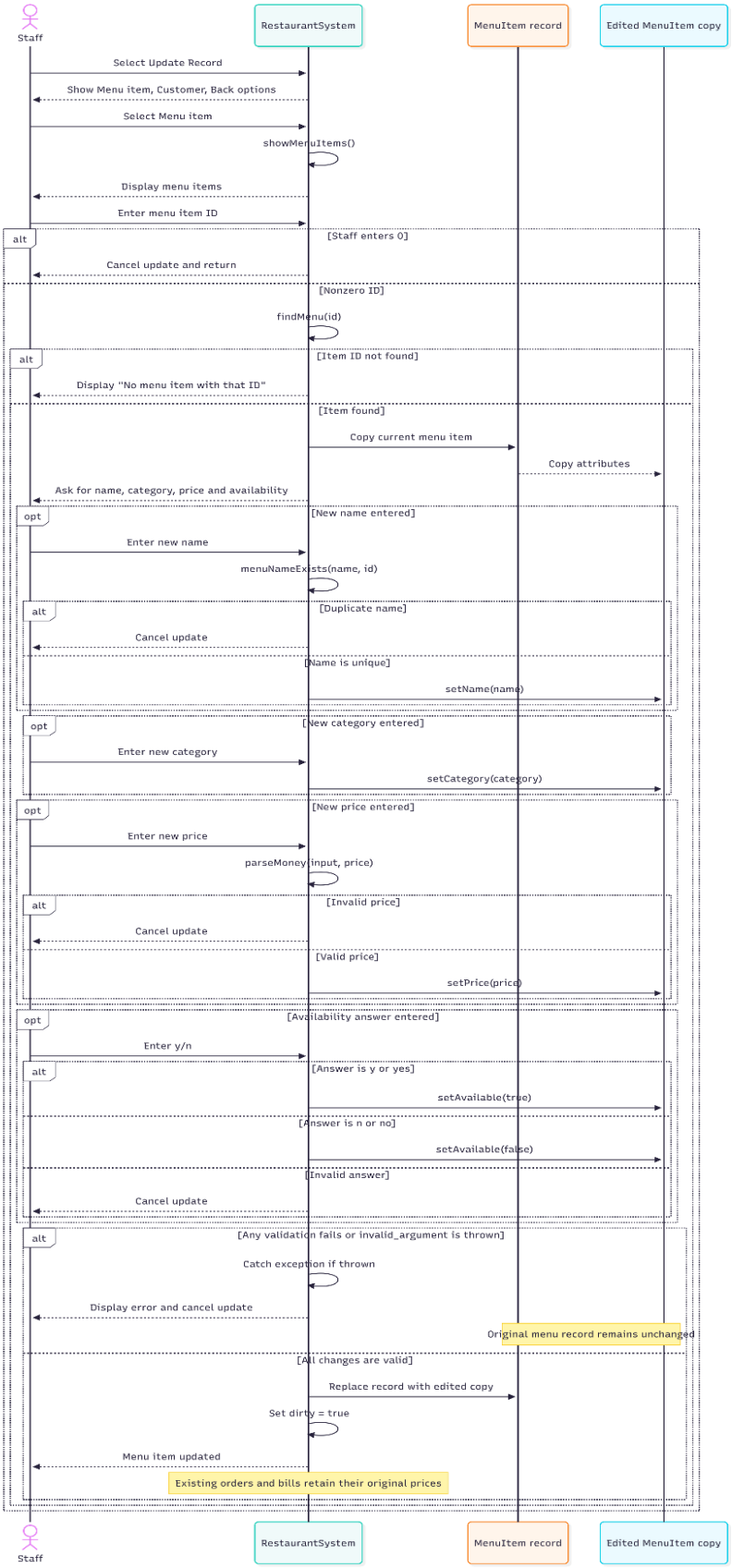
9.2 Class Diagram

Shows the project's classes, their attributes and methods, and the relationships between them.



9.3 Sequence Diagram 1 — Main Transaction

Shows the sequence of interactions involved in creating an order, generating a bill, and optionally recording payment.


Staff
RestaurantSystem
MenuItem record
Edited MenuItem copy
Select Update Record
Show Menu item, Customer, Back options
Select Menu item
showMenuItems()
Display menu items
Enter menu item ID
alt
[Staff enters 0]
Cancel update and return
[Nonzero ID]
findMenu(id)
alt
[Item ID not found]
Display "No menu item with that ID"
[Item found]
Copy current menu item
Copy attributes
Ask for name, category, price and availability
opt
[New name entered]
Enter new name
menuNameExists(name, id)
alt
[Duplicate name]
Cancel update
[Name is unique]
setName(name)
opt
[New category entered]
Enter new category
setCategory(category)
opt
[New price entered]
Enter new price
parseMoney(input, price)
alt
[Invalid price]
Cancel update
[Valid price]
setPrice(price)
opt
[Availability answer entered]
Enter y/n
alt
[Answer is y or yes]
setAvailable(true)
[Answer is n or no]
setAvailable(false)
[Invalid answer]
Cancel update
alt
[Any validation fails or invalid_argument is thrown]
Catch exception if thrown
Display error and cancel update
Original menu record remains unchanged
[All changes are valid]
Replace record with edited copy
Set dirty = true
Menu item updated
Existing orders and bills retain their original prices
Staff
RestaurantSystem
MenuItem record
Edited MenuItem copy

9.4 Sequence Diagram 2 — Menu Item Update

Shows how a menu item is selected, its details are updated, and the system processes the change.

10. Program Workflow

The Restaurant Billing & Payment System follows a menu-driven workflow. When the application starts, it loads the existing records from the data files and displays a main menu through which the user can select the required operation.

Step 1: Start the Application

The user launches Project.exe. The application initializes the restaurant system and loads the available records from the Data directory.

Step 2: Display the Main Menu

The program displays a numbered menu containing options for adding, displaying, searching, updating, and deleting records, performing the main transaction, viewing reports, saving or loading data, and exiting the application.

Step 3: Select an Operation

The user enters the number corresponding to the required operation. The application processes the selection and requests additional information where necessary.

Step 4: Manage Records

The user can perform record-management operations on the supported data, such as menu items and customers. The application processes the requested operation and displays the result.

Step 5: Perform the Main Transaction

When the main transaction option is selected, the system allows the user to create an order by selecting a customer, choosing available menu items, and entering quantities. The system calculates the subtotal and applies the entered discount to generate the bill.

Step 6: Process Payment

The user can record payment through cash, card, or UPI. Alternatively, the bill can remain unpaid for later payment. When payment is successfully recorded, the system updates the bill's payment status and the associated order status.

Step 7: View Reports

The user can select the report option to view the information provided by the application's reporting functionality.

Step 8: Save and Load Data

The application provides save/load functionality to write records to or read records from the text files in the Data directory. This supports the persistence of information between program executions.

Step 9: Exit the Application

When the user selects the exit option, the program follows its implemented exit and saving behaviour and terminates. The user can then reopen the application to verify that saved records remain available.

Workflow Summary

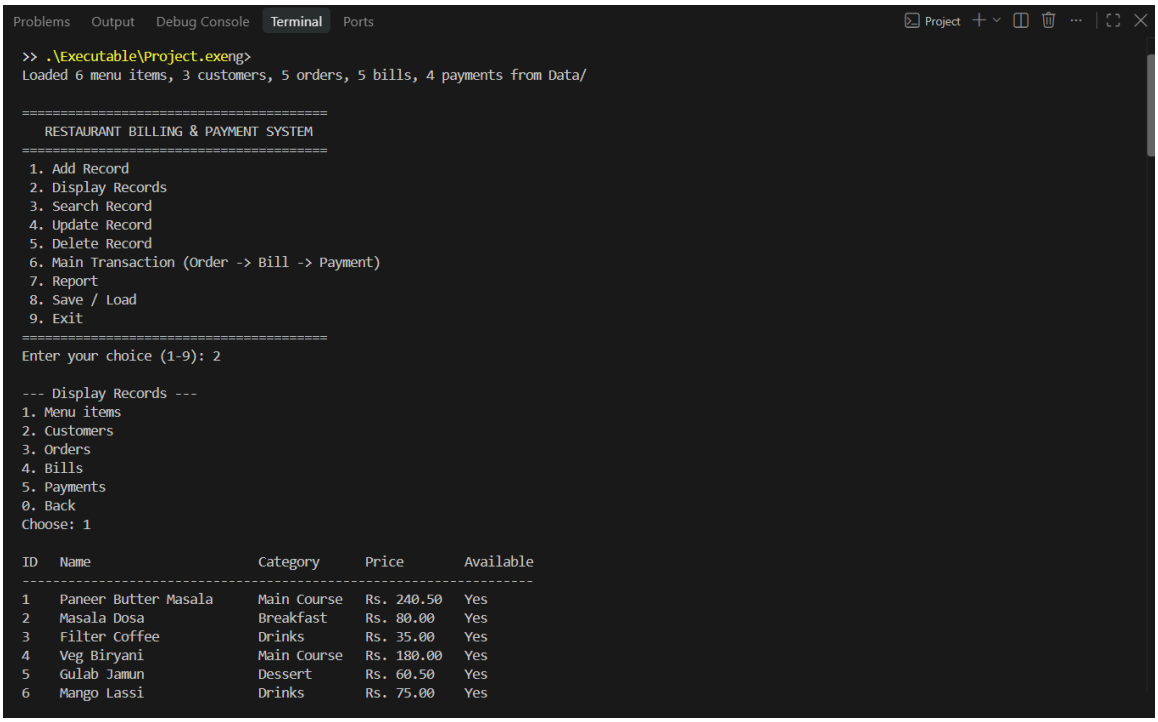
Start Application → Load Existing Data → Display Main Menu → Select Operation → Manage Records or Perform Transaction → View Reports or Save/Load Data → Exit.

This workflow demonstrates how the application's menu, entity classes, transaction processing, payment handling, and file management work together to support restaurant billing operations.

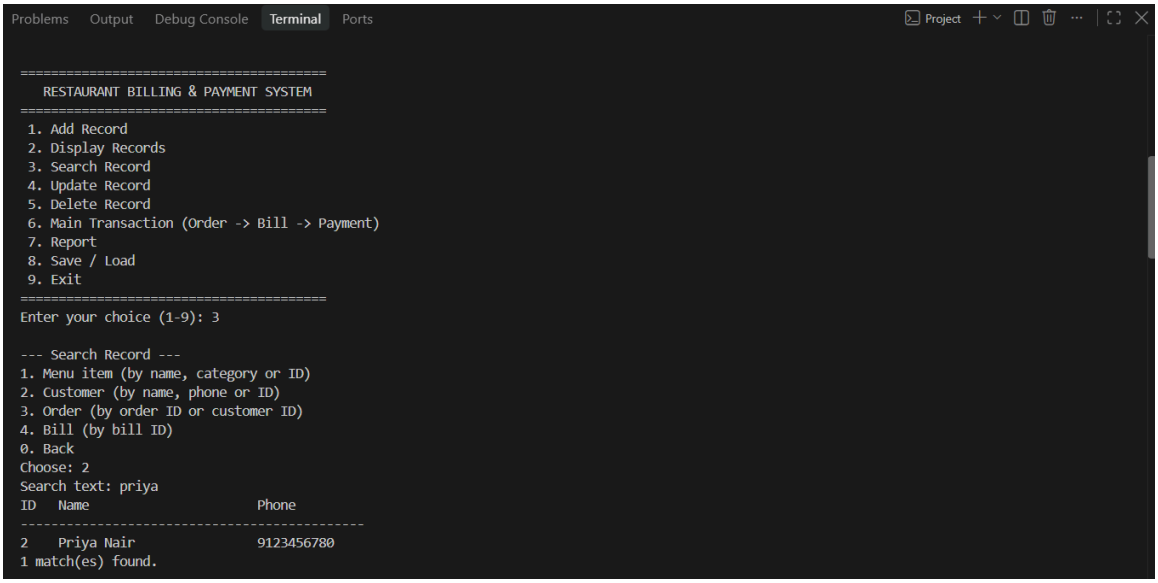
11. Screenshots



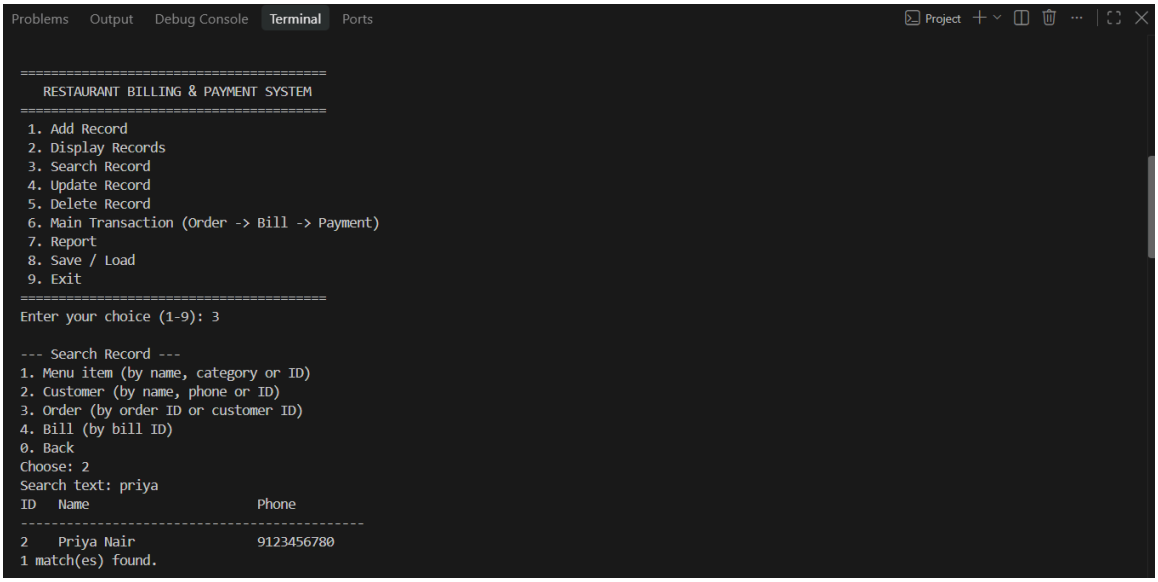
Main Menu



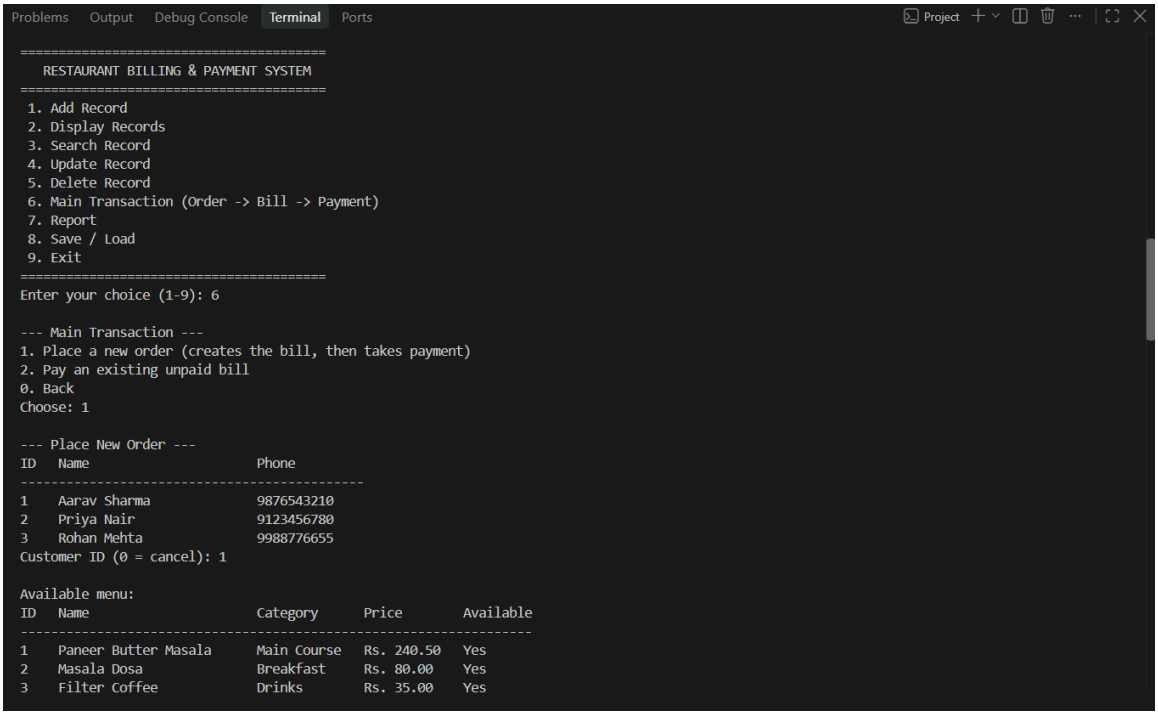
Display Records



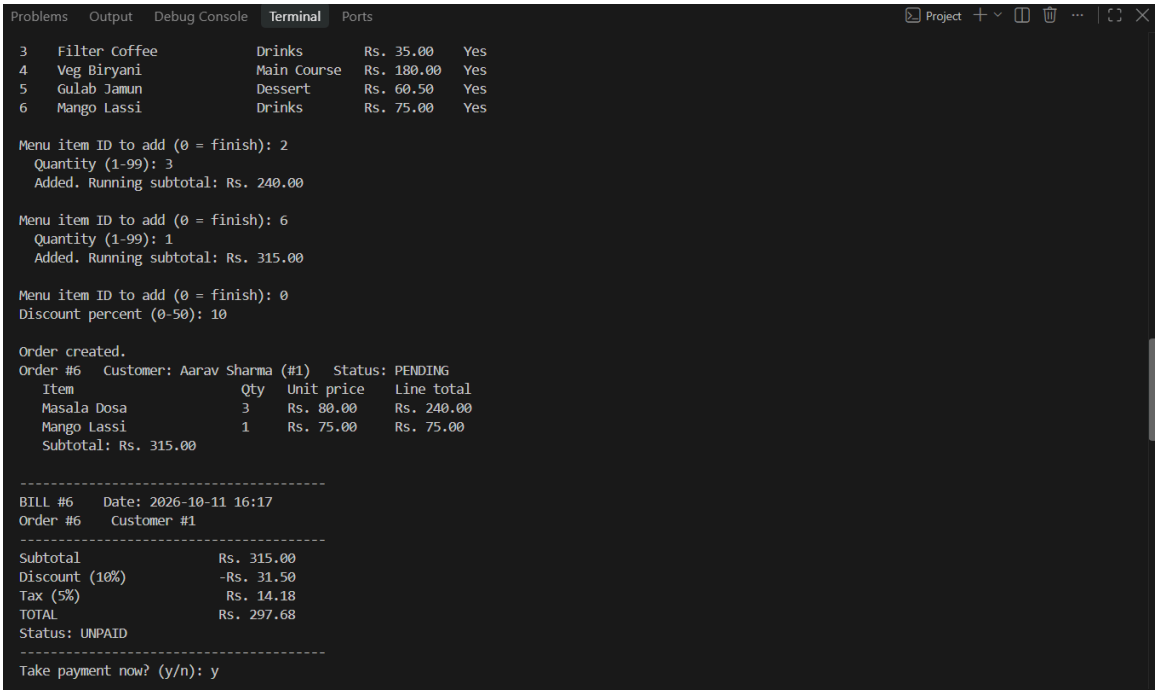
Search Records



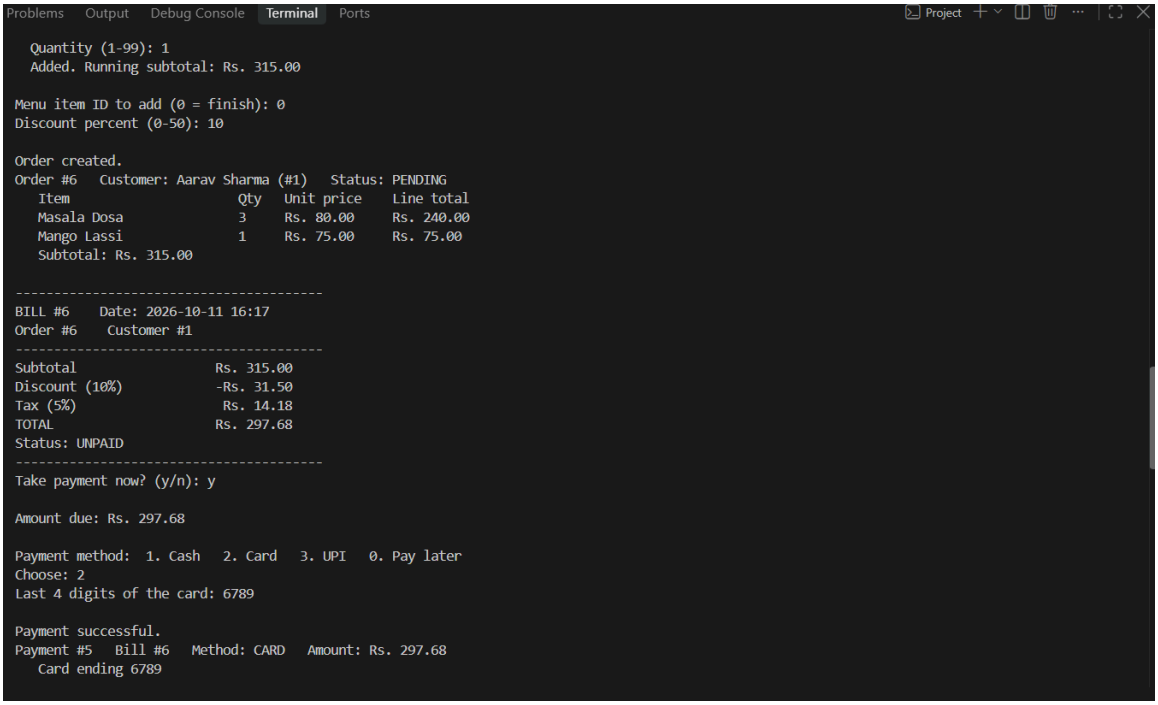
Updated Records



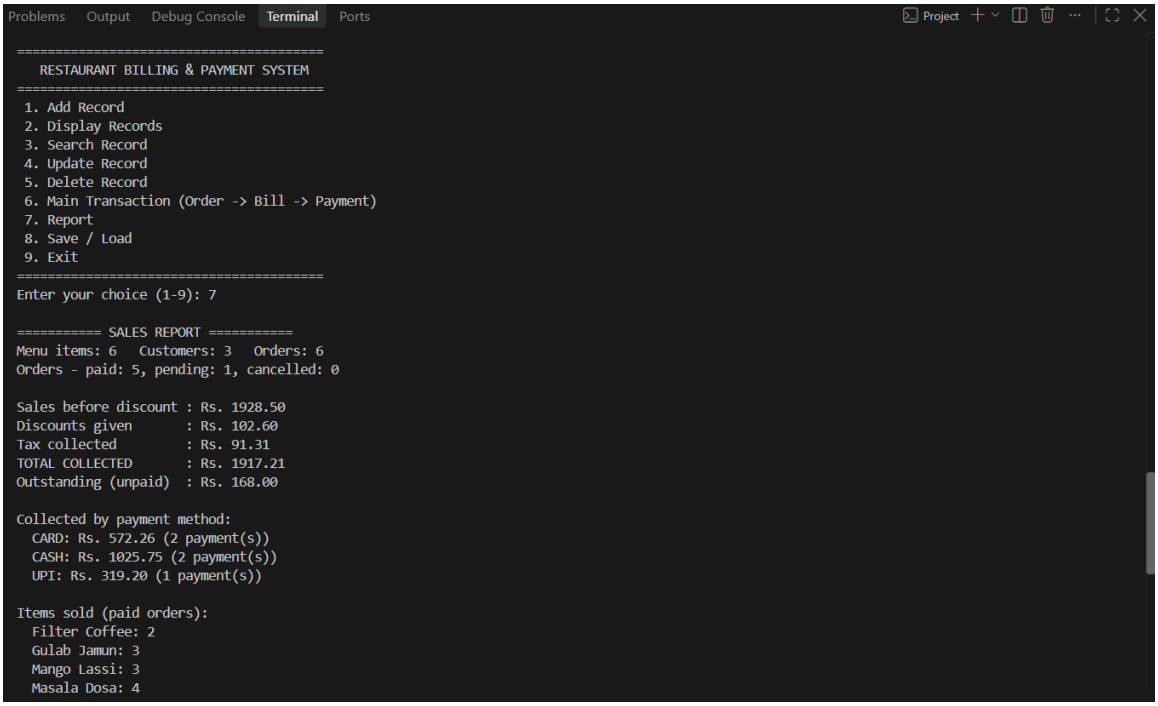
Main Transaction 1



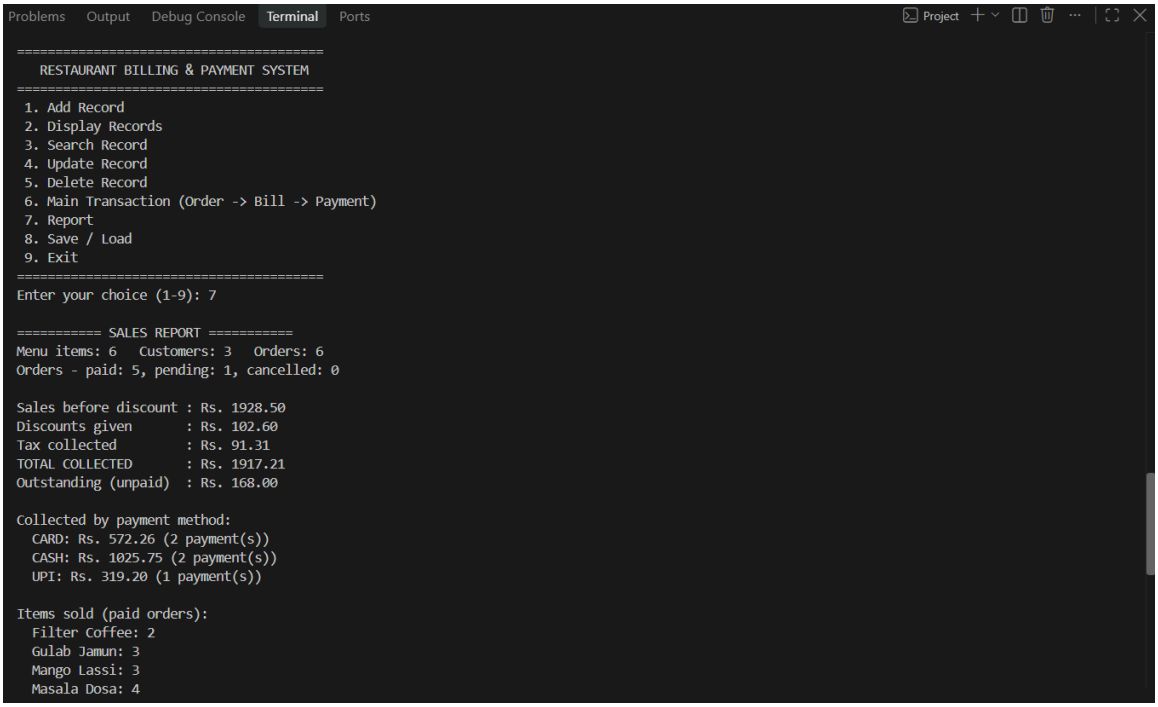
Main Transaction 2



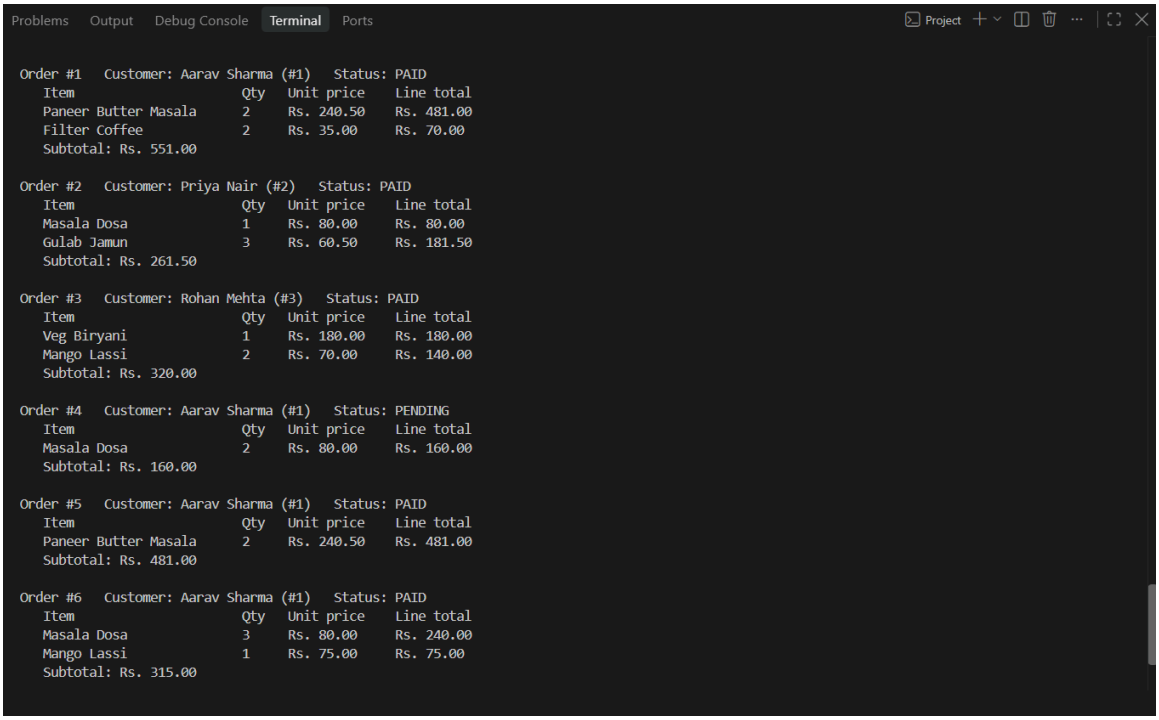
Main Transaction 3



Sales Report



Order Display



Saving Records

12. Testing

Testing was performed to check whether the Restaurant Billing & Payment System compiled successfully, launched correctly, loaded existing records, and performed its main operations. The following test cases describe the observed results and additional checks required before final submission.

12.1 Test Cases

Test Case	Input / Action	Expected Result	Status
Compilation	Run .\build.bat	The program compiles successfully and generates the executable.	Pass

Application Startup	Run .\Executable\Project.exe	The application starts and displays the main menu.	Pass
Loading Existing Records	Start the application with the existing Data files.	Existing menu items, customers, orders, bills, and payments are loaded.	Pass
Search Record	Search for an existing record.	The matching record is displayed.	Verify before submission
Update Menu Item	Change the price of an existing menu item.	The updated details are displayed correctly.	Verify after restoring the original sample data
Add and Display Records	Add a record and display the records.	The new record is stored and displayed correctly.	Verify before submission
Delete Record	Delete an eligible record.	The selected record is removed according to the application's implemented behaviour.	Verify before submission
Main Transaction	Create an order by selecting a customer, menu items, and quantities.	The system calculates the subtotal and generates a bill.	Verify before submission
Payment Processing	Record a payment using a supported payment method.	The payment is recorded and the relevant status is updated.	Verify before submission

Save and Reload	Save changes, close the program, and reopen it.	Saved records remain available after restarting.	Verify before submission

12.2 Observed Startup Results

During a successful application launch, the program displayed the following record counts:

- Menu items: 6
- Customers: 3
- Orders: 4
- Bills: 4
- Payments: 3

These results confirmed that the application could load the existing records from the Data directory at startup.

12.3 Testing Conclusion

The successful build and startup tests confirmed that the project could be compiled and executed in the target environment and that existing records could be loaded. The remaining test cases should be performed using the final submission folder before the project is submitted.

Any failures discovered during testing should be corrected and the affected tests repeated. Only mark a test as passed after observing its expected result.

13. Challenges and Solutions

During the development of the Restaurant Billing & Payment System, several programming and project-management challenges had to be addressed. These challenges helped improve the reliability, organization, and usability of the application.

13.1 Organizing the Program into Classes

Challenge: Managing menu items, customers, orders, bills, and payments in one program could make the code difficult to understand and maintain.

Solution: The application uses separate classes for different responsibilities. This organizes related data and operations together and makes the program more modular.

13.2 Managing Orders and Order Items

Challenge: An order can contain multiple menu items, each with its own quantity and price. Keeping this information organized is necessary for accurate billing.

Solution: The Order class uses OrderLine objects to store individual item details. The order subtotal is calculated from the stored order lines.

13.3 Supporting Different Payment Methods

Challenge: The application needs to represent different payment methods without duplicating the entire payment-processing structure.

Solution: The abstract Payment class provides a common interface, while CashPayment, CardPayment, and UpiPayment implement the supported payment methods.

13.4 Storing Data Between Executions

Challenge: Records entered during one execution should remain available after the program closes.

Solution: The project uses text files in the Data directory. The FileManager class provides file-reading and file-writing operations, allowing the application to load and save records.

13.5 Handling Invalid Inputs

Challenge: Invalid input, such as an incorrect menu selection or an unacceptable value, can interrupt normal program operations or lead to incorrect results.

Solution: The application uses input validation and exception handling in relevant operations to reject invalid values and handle errors appropriately.

13.6 Building and Organizing the Submission

Challenge: A complete college project requires more than source code. It also requires data files, diagrams, screenshots, documentation, and a working executable.

Solution: The project is organized into separate folders for source code, data, diagrams, screenshots, and the executable. A build script is also provided to compile the program.

13.7 Lessons Learned

Working on this project provided practical experience in object-oriented programming, file handling, transaction processing, debugging, testing, and project documentation. It also highlighted the importance of checking the program using realistic inputs and verifying that saved data remains available after restarting the application.

14. Conclusion

The Restaurant Billing & Payment System was developed as a C++ project to demonstrate the practical application of object-oriented programming, file handling, and transaction processing.

The application provides functionality for managing restaurant records, handling customer orders, generating bills, and recording payments through supported payment methods. It uses classes to organize different responsibilities and demonstrates concepts such as encapsulation, inheritance, runtime polymorphism, dynamic memory management, and exception handling.

The use of text files allows application records to be stored and loaded between program executions. The modular project structure, UML diagrams, screenshots, and test cases provide supporting documentation of the system's design and implementation.

Through this project, practical experience was gained in writing modular C++ code, working with classes and objects, managing files, compiling and executing a program, testing functionality, and preparing technical documentation.

Overall, the project demonstrates how C++ and object-oriented programming concepts can be applied to develop a basic restaurant billing and payment application. Further improvements could include a graphical user interface, database integration, enhanced security, and more advanced reporting features.

15. Github Repository Link

https://github.com/janiraina29-gif/RestaurantBilling